LINKED LIST

Foundation & Templates

>

WHY LINKED LISTS?

Arrays require contiguous memory blocks. Linked Lists store nodes anywhere in Heap memory, linked via **next pointers** for **O(1) insertion/deletion** without shifting!

THE DUMMY NODE AHA

Always use a **Dummy Head Sentinel Node** (`ListNode dummy = new ListNode(0)`) to eliminate edge cases when modifying or removing the real head node!

1 CORE NODE STRUCTURE

```
public class ListNode {
  int val;
  ListNode next;
  ListNode(int val) { this.val = val;
}
  ListNode(int val, ListNode next) {
    this.val = val; this.next = next;
  }
}
```

2 VISUALIZING POINTER REVERSAL

3-Pointer Window (prev, curr, next):

```
prev = null,  curr = [1] -> [2] ->
[3] -> null
Step 1: next = curr.next ([2])
Step 2: curr.next = prev (null <-
[1])
Step 3: prev = curr ([1]), curr = nex
t ([2])
Final Reversed List: [3] -> [2] ->
[1] -> null
```

PREREQUISITES & COMPLEXITY REASONING

Operation	Time Complexity	Auxiliary Space
Prepend / Append (with tail pointer)	**O(1)**	**O(1)**
Search / Index Access	**O(N)**	**O(1)**
In-place Reversal	**O(N)**	**O(1)**
Fast / Slow Cycle Detection	**O(N)**	**O(1)**

CLUES THAT SCREAM LINKED LIST

1. *"Modify list in-place"* \rightarrow 3-Pointer Reversal or Fast/Slow Pointers
2. *"Find cycle / entry point"* \rightarrow Floyd's Tortoise and Hare
3. *"Remove Nth node from end"* \rightarrow 2 Pointers separated by offset N
4. *"Merge K sorted lists"* \rightarrow Min-Heap PriorityQueue O(N \log K)

>

1 DUMMY SENTINEL PATTERN (`ListNodeUtils`)

```
public class ListNodeUtils {
  // Always attach dummy head when building or filtering list
s
  public static ListNode createList(int[] values) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    for (int val : values) {
      curr.next = new ListNode(val);
      curr = curr.next;
    }
    return dummy.next; // Real head
  }
}
```

2 IN-PLACE REVERSAL HELPER

```
public static ListNode reverse(ListNode head) {
  ListNode prev = null, curr = head;
  while (curr != null) {
    ListNode nextTemp = curr.next;
    curr.next = prev;
    prev = curr;
    curr = nextTemp;
  }
  return prev; // New head of reversed list!
}
```

3 FIND MIDDLE NODE (Fast & Slow Pointers)

```
public static ListNode findMiddle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow; // Returns second middle for even length
}
```

4 MERGE TWO SORTED LISTS

```
public static ListNode merge(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0), curr = dummy;
  while (l1 != null && l2 != null) {
    if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
    else { curr.next = l2; l2 = l2.next; }
    curr = curr.next;
  }
  curr.next = (l1 != null) ? l1 : l2;
  return dummy.next;
}
```

>

PATTERN 1 IN-PLACE POINTER REVERSAL (prev, curr, next)

e.g. Reverse Linked List (LC 206), Reverse Nodes in K-Group (LC 25)

WHEN TO USE

Reversing full or subsegments of a linked list in O(N) time and O(1) auxiliary space.

TEMPLATE — LC 206

```
public ListNode reverseList(ListNode head) {
  ListNode prev = null, curr = head;
  while (curr != null) {
    ListNode next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

PATTERN 2 DUMMY SENTINEL NODE (Head Modification Guard)

e.g. Merge Two Sorted Lists (LC 21), Partition List (LC 86)

WHEN TO USE

Whenever operations might delete, swap, or prepend nodes at the head of the list.

TEMPLATE — LC 21

```
public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
  ListNode dummy = new ListNode(0), curr = dummy;
  while (list1 != null && list2 != null) {
    if (list1.val <= list2.val) { curr.next = list1; list1 = list1.next; }
    else { curr.next = list2; list2 = list2.next; }
    curr = curr.next;
  }
  curr.next = (list1 != null) ? list1 : list2;
  return dummy.next;
}
```

>

PATTERN 3 FLOYD'S TORTOISE & HARE (Cycle Detection & Entry)

e.g. Linked List Cycle I & II (LC 141, 142)

MATHEMATICAL PROOF

`fast` moves at 2x speed of `slow`. If cycle exists, distance reduces by 1 each step until collision.
To find cycle start: reset `slow` to `head`, then move both 1 step at a time until they meet!

TEMPLATE — LC 141

```
public boolean hasCycle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) return true;
  }
  return false;
}
```

PATTERN 4 FAST & SLOW MIDDLE / PALINDROME SPLIT

e.g. Palindrome Linked List (LC 234), Reorder List (LC 143)

3-STEP STRATEGY

1. Find middle via fast/slow.
2. Reverse second half.
3. Compare / interleave two halves!

TEMPLATE — LC 234

```
public boolean isPalindrome(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next; fast = fast.next.next;
  }
  ListNode prev = null, curr = slow;
  while (curr != null) {
    ListNode nxt = curr.next; curr.next = prev; prev = curr; curr = nxt;
  }
  while (prev != null) {
    if (head.val != prev.val) return false;
    head = head.next; prev = prev.next;
  }
```

>

PATTERN 5 TWO POINTER OFFSET SEARCH (Nth Node From End) e.g. Remove Nth Node From End (LC 19)

OFFSET MECHANICS

Advance `fast` by N + 1 steps ahead of `dummy`. Then move `fast` and `slow` together until `fast == null`. `slow.next` is the N-th node from end!

TEMPLATE — LC 19

```
public ListNode removeNthFromEnd(ListNode head, int n) {
  ListNode dummy = new ListNode(0);
  dummy.next = head;
  ListNode fast = dummy, slow = dummy;
  for (int i = 0; i <= n; i++) fast = fast.next;
  while (fast != null) {
    fast = fast.next;
    slow = slow.next;
  }
  slow.next = slow.next.next;
  return dummy.next;
}
```

PATTERN 6 REORDER LIST INTERLEAVING e.g. Reorder List (LC 143) — $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1}$

INTERLEAVING STRATEGY

Split list at mid, reverse second half, then interleave pointers `first` and `second` step by step!

TEMPLATE — LC 143

```
public void reorderList(ListNode head) {
  if (head == null || head.next == null) return;
  ListNode slow = head, fast = head;
  while (fast.next != null && fast.next.next != null) {
    slow = slow.next; fast = fast.next.next;
  }
  ListNode prev = null, curr = slow.next; slow.next = null;
  while (curr != null) {
    ListNode nxt = curr.next; curr.next = prev; prev = curr; curr = nxt;
  }
  ListNode first = head, second = prev;
  while (second != null) {
    ListNode t1 = first.next, t2 = second.next;
    first.next = second; second.next = t1;
    first = t1; second = t2;
  }
}
```

>

PATTERN 7 MERGE K SORTED LISTS (Min-Heap PriorityQueue) e.g. Merge K Sorted Lists (LC 23)

MIN-HEAP COMPLEXITY

Heap size is at most K. Each insertion/deletion takes O(\log K). Total time $= O(N \log K) where N$ is total nodes!

TEMPLATE — LC 23

```
public ListNode mergeKLists(ListNode[] lists) {
  if (lists == null || lists.length == 0) return null;
  PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b)->a.val-b.val);
  for (ListNode node : lists) if (node != null) pq.offer(node);
  ListNode dummy = new ListNode(0), curr = dummy;
  while (!pq.isEmpty()) {
    ListNode minNode = pq.poll();
    curr.next = minNode;
    curr = curr.next;
    if (minNode.next != null) pq.offer(minNode.next);
  }
  return dummy.next;
}
```

PATTERN 8 COPY LIST WITH RANDOM POINTER (HashMap / Interleaving) e.g. Copy List with Random Pointer (LC 138)

HASHMAP MAPPING STRATEGY

TEMPLATE — LC 138

LINKED LIST

Decision Tree & Trigger Words

>

LINKED LIST — WHICH ALGORITHM TO USE?

Start: What is the operation?

- 🔄 **Reverse list or subsegment** → **3-Pointer Reversal** `prev`, `curr`, `next`
- 🛡️ **Modify/Delete list head** → **Dummy Sentinel** `ListNod dummy new ListNod
- 🌀 **Detect cycle / find loop entry** → **Floyd's Fast/Slow** `fast` moves 2x, `slow` 1x
- 🔪 **Find Nth node from end** → **Offset 2-Pointer** `fast` advanced by N+1
- 🔀 **Merge K sorted lists** → **Min-Heap PQ** PriorityQueue of heads

🎯 TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Reverse in-place"	3-Pointer	Loop `curr.next = prev`
"Detect cycle"	Floyd's Fast/Slow	Check `slow == fast`
"Nth node from end"	Offset Pointers	Advance `fast` by N+1
"Merge K sorted"	Min-Heap PQ	Offer heads to PriorityQueue

💼 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why use Dummy Node?	Eliminates conditional special-casing when updating or deleting the head node!
Array vs Linked List insertion?	Array insert takes O(N) shift time; Linked List insert takes O(1) given pointer node!

LINKED LIST

FAANG Problem Ladder (All 10 Problems)

>

EASY — Fundamental Reversals & Cycles

LC 206 · Reverse Linked List — EASY

Pattern: 3-Pointer Reversal
Key Insight: Save `curr.next` before overwriting.

```
public ListNode reverseList(ListNode head) {
  ListNode prev = null, curr = head;
  while (curr != null) {
    ListNode nxt = curr.next;
    curr.next = prev;
    prev = curr;
    curr = nxt;
  }
  return prev;
}
```

LC 21 · Merge Two Sorted Lists — EASY

Pattern: Dummy Sentinel
Key Insight: Append smaller node. Attach remaining.

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0), curr = dummy;
  while (l1 != null && l2 != null) {
    if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
    else { curr.next = l2; l2 = l2.next; }
    curr = curr.next;
  }
  curr.next = (l1 != null) ? l1 : l2;
  return dummy.next;
}
```

LC 141 · Linked List Cycle — EASY

Pattern: Floyd's Fast / Slow
Key Insight: `fast` moves 2x, `slow` 1x. Collision = Cycle!

```
public boolean hasCycle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next; fast = fast.next.next;
    if (slow == fast) return true;
  }
  return false;
}
```

MEDIUM — Palindromes & Nth Node Removal

LC 234 · Palindrome Linked List — EASY

Pattern: Mid Split + Reversal
Key Insight: Find mid, reverse second half, compare!

```
public boolean isPalindrome(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next; fast = fast.next.next;
  }
  ListNode prev = null, curr = slow;
  while (curr != null) {
    ListNode nxt = curr.next; curr.next = prev; prev = curr; curr = nxt;
  }
  while (prev != null) {
    if (head.val != prev.val) return false;
    head = head.next; prev = prev.next;
  }
  return true;
}
```

LC 19 · Remove Nth From End — MEDIUM

Pattern: Offset Pointers
Key Insight: Advance `fast` by N+1 from `dummy`.

```
public ListNode removeNthFromEnd(ListNode head, int n) {
  ListNode dummy = new ListNode(0); dummy.next = head;
  ListNode fast = dummy, slow = dummy;
  for (int i = 0; i <= n; i++) fast = fast.next;
  while (fast != null) { fast = fast.next; slow = slow.next; }
  slow.next = slow.next.next;
  return dummy.next;
}
```

>

MEDIUM — Reorder List & Add Two Numbers

LC 143 · Reorder List — MEDIUM

Pattern: Mid + Reverse + Interleave
Key Insight: Interleave first and reversed second.

```
public void reorderList(ListNode head) {
  if (head == null || head.next == null) return;
  ListNode s = head, f = head;
  while (f.next != null && f.next.next != null) { s = s.next; f = f.next.next; }
  ListNode prev = null, curr = s.next
```

HARD — Merge K Lists & Random Clone

LC 138 · Copy Random List — MEDIUM

Pattern: HashMap Clone
Key Insight: Map `old -> new` in Pass 1, link in Pass 2.

```
public Node copyRandomList(Node head) {
  if (head == null) return null;
  Map<Node, Node> map = new HashMap<>();
  Node curr = head;
  while (curr != null) { map.put(curr, new Node(curr.val)); curr = curr.next; }
```

>

🔍 DRY RUN — Reverse Linked List ([1 -> 2 -> 3])

Step	prev	curr	next	Action / Pointer Assignment
1	null	[1]	[2]	`curr.next = prev` (1 -> null). `prev = [1], curr = [2]`
2	[1]	[2]	[3]	`curr.next = prev` (2 -> 1). `prev = [2], curr = [3]`
3	[2]	[3]	null	`curr.next = prev` (3 -> 2). `prev = [3], curr = null`
End	[3]	null	-	Return `prev` ([3] is new head!) ✅

📐 MATHEMATICAL PROOF — FLOYD'S CYCLE DETECTION

Let L be distance from head to cycle entrance, C cycle length, K distance from entrance to collision point.

Slow distance $= L + K. Fast distance = L + K + nC = 2(L + K)$.

Thus, $L + K = nC => L = nC - K$. Resetting `slow` to head and moving both 1 step at a time guarantees meeting at cycle entrance after L steps!

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write 3-Pointer Reversal in 2m
☐ Use Dummy Sentinel node for all head ops
☐ Implement Floyd's Fast/Slow cycle check
☐ Code Remove Nth Node From End
☐ Code Palindrome Linked List in O(1) space
☐ Write Reorder List (Mid + Reverse + Interleave)
☐ Code Merge K Sorted Lists with Min-Heap
☐ Prove Floyd's Cycle Entry math on board

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Guard Against Null Pointer

Before accessing `curr.next.val`, always check `if (curr != null && curr.next != null)`!

Rule 2 — Disconnect Broken Links

When splitting lists (e.g., Reorder List), set `slow.next = null` to avoid infinite cycle traps!